

```

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Date;
import java.util.GregorianCalendar;

package nhis.fw.commons.util;

}

return defaultString(String.valueOf(arg).trim(), defaulttValue);
}

if (arg == null) {
    return defaulttValue;
}

public static String defaultStringTrim(Object arg, String defaulttValue) {
    /**
     * @param defaulttValue
     * @param arg
     * @return String
     */
    return defaultString(String.valueOf(arg).trim(), defaulttValue);
}

return str;
}

if ("".equals(str)) {
    return defaulttValue;
}

String str = String.valueOf(arg);

if (arg == null) {
    return defaulttValue;
}

public static String defaultStringTrim(Object arg, String defaulttValue) {
    /**
     * @param defaulttValue
     * @param arg
     * @return String
     */
    return defaultString(String.valueOf(arg).trim(), defaulttValue);
}

return unexponential(defaultString(value, defaulttValue));
}

public static String unexponential(Object value, String defaulttValue) {
    /**
     * @param value
     * @param defaulttValue
     * @return String
     */
    return unexponential(String.valueOf(value).trim(), defaulttValue);
}

return number;
}

```

```

import java.util.Locale;
import java.util.TimeZone;

public class CalendarWrapper {
    private transient GregorianCalendar cal;

    private static SimpleDateFormat textFormatter = new SimpleDateFormat("yyyy년 MM월 dd일", Locale.KOREA);
    private static SimpleDateFormat textTimeFormatter = new SimpleDateFormat("HH:mm:ss", Locale.KOREA);

    private static TimeZone tz = TimeZone.getTimeZone("Asia/Seoul");

    static {
        textFormatter.setTimeZone(tz);
        textTimeFormatter.setTimeZone(tz);
    }

    private static final String[] DOM_NMS_KR = new String[] {"일요일", "월요일", "화요일", "수요일", "목요일", "금요일", "토요일", "일요일"};
    private static final String[] DOM_NMS_EN = new String[] {"SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"};

    public CalendarWrapper() {
        this(new Date(System.currentTimeMillis()));
    }

    public CalendarWrapper(String strDate) throws IllegalArgumentException {
        this.cal = toCalendar(strDate);
    }

    public CalendarWrapper(Date date) {
        this.cal = new GregorianCalendar(tz, Locale.KOREA);
        this.cal.setTime(date);
    }

    public CalendarWrapper(long time) {
        this.cal = new GregorianCalendar(tz, Locale.KOREA);
        this.cal.setTimeInMillis(time);
    }

    public CalendarWrapper(int year, int month, int day) {
        this.cal = new GregorianCalendar(year, month - 1, day);
        this.cal.setTimeZone(tz);
    }

    public CalendarWrapper(int year, int month, int day, int hour, int min, int sec) {
        this.cal = new GregorianCalendar(year, month - 1, day, hour, min, sec);
        this.cal.setTimeZone(tz);
    }

    public String getDateAsString(String delim) {
        SimpleDateFormat sdf = getFormatter("yyyy" + delim + "MM" + delim + "dd");
        return sdf.format(getDate());
    }

    public String getTimeAsString(String delim) {
        SimpleDateFormat sdf = getFormatter("HH" + delim + "mm" + delim + "ss");
        return sdf.format(getDate());
    }
}

```

```

public String getDate(String dateIm, String timeDelim) {
    SimpleDateFormat sdf = getFormatter()
        new
        StringBulider("yyyy").append(dateIm).append("MM").append("ss").toString());
        return sdf.format(getDate());
    }

    public String getDateAsString() {
        return getDateAsString();
    }

    public String getTimeAsString() {
        return getTimeAsString();
    }

    public String getDateAsString() {
        return String.valueOf(getBlankDateString()) + getBlankTimeString();
    }

    public String getDateAsString() {
        SimpleDateFormat sdf = getFormatter("yyMMdd");
        return sdf.format(getDate());
    }

    public String getDashDateString() {
        SimpleDateFormat sdf = getFormatter("yyyy-MM-dd");
        return sdf.format(getDate());
    }

    public String getSlashDateString() {
        SimpleDateFormat sdf = getFormatter("yyyy/MM/dd");
        return sdf.format(getDate());
    }

    public String getDotDateString() {
        SimpleDateFormat sdf = getFormatter("yyyy.MM.dd");
        return sdf.format(getDate());
    }

    public String getBlankTimeString() {
        SimpleDateFormat sdf = getFormatter("HHmmss");
        return sdf.format(getDate());
    }

    public String getColonTimeString() {
        SimpleDateFormat sdf = getFormatter("HH:mm:ss");
        return sdf.format(getDate());
    }

    public String getDateAsString() {
        synchronized (textFormatter) {
            return textFormatter.format(getDate());
        }
    }

    public String getTimeAsString() {
        synchronized (textFormatter) {
            return textFormatter.format(getDate());
        }
    }

    public String getTimeAsString() {
        synchronized (textFormatter) {
            return textFormatter.format(getDate());
        }
    }
}

```



```

    public String getDateTimeString() {
        synchronized (textFormatter) {
            return String.valueOf(textFormatter.format(getDate())) + " " +
                textFormatter.format(getDate());
        }
    }

    private GregorianCalendar toCalendar(String strDate) throws IllegalArgumentException {
        if (StringUtil.isEmpty(strDate))
            throw new IllegalArgumentException("Date String Format Error : " + strDate);
        String tempStrDate = strDate;
        if (tempStrDate == null)
            tempStrDate = "";
        tempStrDate = tempStrDate.replaceAll("\\D", "");
        int len = tempStrDate.length();
        if (len < 8)
            throw new IllegalArgumentException("Date String Format Error : " +
                tempStrDate);
        int year = Integer.parseInt(tempStrDate.substring(0, 4));
        int mon = Integer.parseInt(tempStrDate.substring(4, 6));
        int day = Integer.parseInt(tempStrDate.substring(6, 8));
        int hour = 0;
        int min = 0;
        int sec = 0;
        int millis = 0;
        if (len >= 17) {
            millis = Integer.parseInt(tempStrDate.substring(14, 17));
            sec = Integer.parseInt(tempStrDate.substring(12, 14));
            min = Integer.parseInt(tempStrDate.substring(10, 12));
            hour = Integer.parseInt(tempStrDate.substring(8, 10));
        } else if (len >= 10) {
            hour = Integer.parseInt(tempStrDate.substring(8, 10));
        }
        if (len >= 14) {
            sec = Integer.parseInt(tempStrDate.substring(12, 14));
        }
        min = Integer.parseInt(tempStrDate.substring(10, 12));
        hour = Integer.parseInt(tempStrDate.substring(8, 10));
        if (len >= 12) {
            min = Integer.parseInt(tempStrDate.substring(10, 12));
        } else if (len >= 12) {
            min = Integer.parseInt(tempStrDate.substring(10, 12));
        }
        min = Integer.parseInt(tempStrDate.substring(10, 12));
    }

    }

    GregorianCalendar cal = new GregorianCalendar(year, mon - 1, day, hour, min, sec);
    cal.setTimeZone(tz);
    return cal;
}

public int getYear() {
    return this.cal.get(1);
}

public int getMonth() {
    return this.cal.get(2) + 1;
}

public int getDay() {
    return this.cal.get(5);
}

public int getHour() {
    return this.cal.get(11);
}
}

```

```

    public int getMinute() {
        return this.cal.get(12);
    }

    public int getSecond() {
        return this.cal.get(13);
    }

    public int getMilliscond() {
        return this.cal.get(14);
    }

    public Date afterDate(int year, int month, int day) {
        GregorianCalendar aca1 = (GregorianCalendar)this.cal.clone();
        aca1.add(1, year);
        aca1.add(2, month);
        aca1.add(5, day);
        return new Date(aca1.getTimeInMillis());
    }

    public Date afterDate(int year, int month, int day, int hour, int min, int sec) {
        GregorianCalendar aca1 = (GregorianCalendar)this.cal.clone();
        aca1.add(1, year);
        aca1.add(2, month);
        aca1.add(5, day);
        aca1.add(11, hour);
        aca1.add(12, min);
        aca1.add(13, sec);
        return new Date(aca1.getTimeInMillis());
    }

    public Date beforeDate(int year, int month, int day) {
        return afterDate(-year, -month, -day);
    }

    public Date beforeDate(int year, int month, int day, int hour, int min, int sec) {
        return afterDate(-year, -month, -day, -hour, -min, -sec);
    }

    public Date afterYear(int year) {
        return add(1, year);
    }

    public Date afterMonth(int month) {
        return add(2, month);
    }

    public Date afterDay(int day) {
        return add(5, day);
    }

    public Date beforeYear(int year) {
        return add(1, -year);
    }

    public Date beforeMonth(int month) {
        return add(2, -month);
    }

    public Date beforeDay(int day) {
        return add(5, -day);
    }
}

```

```

private Date add(int field, int amount) {
    GregorianCalendar aCal = (GregorianCalendar)this.cal.clone();
    aCal.add(field, amount);
    return new Date(aCal.getTimeInMillis());
}

public boolean isAfter(Date aDate) {
    Calendar aCal = (GregorianCalendar)this.cal.clone();
    aCal.setTime(aDate);
    return this.cal.after(aCal);
}

public int getDayOfWeek() {
    return this.cal.get(7) - 1;
}

public String getDayOfWeekInKorean() {
    int dayOfWeek = this.cal.get(7);
    String[] DOM_NMS_KR_ARRAY = getPrivatArray(DOM_NMS_KR);
    return DOM_NMS_KR_ARRAY[dayOfWeek - 1];
}

public String getDayOfWeekInEnglish() {
    int dayOfWeek = this.cal.get(7);
    String[] DOM_NMS_EN_ARRAY = getPrivatArray(DOM_NMS_EN);
    return DOM_NMS_EN_ARRAY[dayOfWeek - 1];
}

public int getDayOfMonth() {
    return this.cal.getActualMaximum(5);
}

public String[] getDatesOfWeeks() {
    String dateStr = getBlankDateString();
    return getWeekDays(dateStr);
}

public boolean isLeapYear() {
    int year = getYear();
    return isLeapYear(year);
}

public long getTimeInMillis() {
    return getDate().getTimeInMillis();
}

public Date getDate() {
    long timeInMillis = this.cal.getTimeInMillis();
    return new Date(timeInMillis);
}

private SimpleDateFormat getFormatter(String pattern) {
    SimpleDateFormat sdf = new SimpleDateFormat(pattern, Locale.KOREA);
    sdf.setTimeZone(tz);
    return sdf;
}

public String toString() {
    Date date = this.cal.getTime();
    return date.toString();
}

```



```
private static final int[] months = new int[] {
    0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 31, 30, 31, 31, 30, 31, 30, 31};
private static final int[] monthStacks = new int[] {
    0, 0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365};
private static final int[] reverseMonthStacks = new int[] {
    0, 365, 334, 306, 275, 245, 214, 184, 153, 122, 92, 61, 31};
private static final int[] monthLeapYear = new int[] {
    0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
private static final int[] monthStacksLeapYear = new int[] {
    0, 0, 31, 60, 91, 121, 152, 182, 213, 244, 274, 305, 335, 366};
private static final int[] reverseMonthStacksLeapYear = new int[] {
    0, 366, 335, 306, 275, 245, 214, 184, 153, 122, 92, 61, 31};
private static int getLeapYearCount(int year) {
    if (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0))
        return 1;
    return 0;
}
private static boolean isLeapYear(int year) {
    if (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0))
        return true;
    return false;
}
private static String getDateDaysAfter(int afterVal, int yearVal, int monthVal, int dayVal) {
    int after = afterVal;
    int year = yearVal;
    int month = monthVal;
    int day = dayVal;
    if (!isLeapYear(year)) {
        if (after <= months[month] - day)
            return Integer.toString(year * 10000 + month * 100 + day + after);
        after -= months[month];
        month++;
        if (month == 13) {
            year++;
            month = 1;
        }
    } else {
        if (after <= monthsLeapYear[month] - day)
            return Integer.toString(year * 10000 + month * 100 + day + after);
        after -= monthsLeapYear[month];
        month++;
        if (month == 13) {
            year++;
            month = 1;
        }
    }
}
```